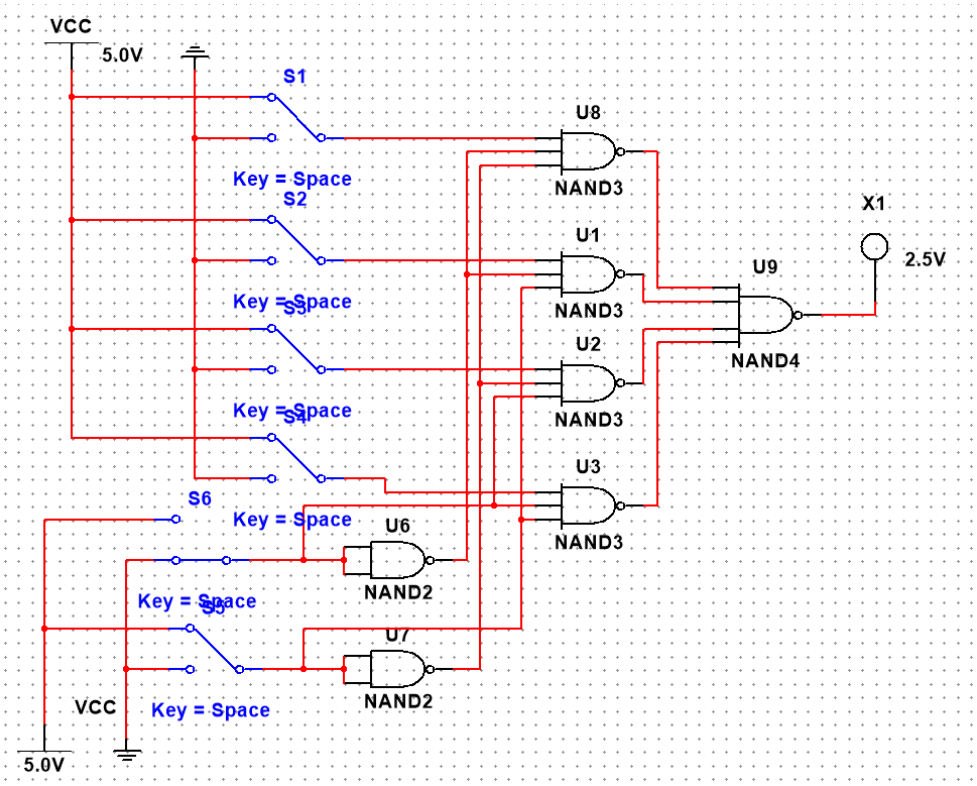
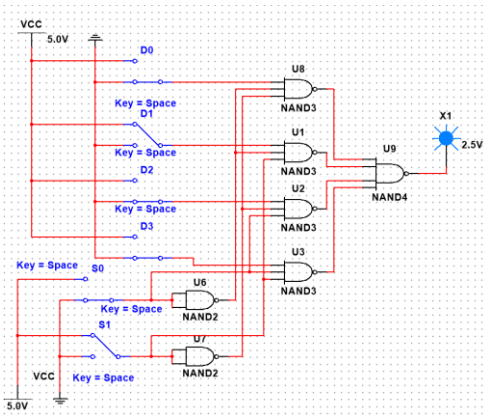
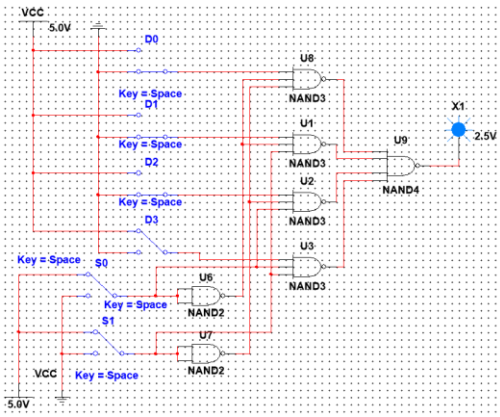
Simulation-Based Tutorial 3

Aim:

The aim of this report is to design, implement, and test 3 different circuits: A four-bit digital multiplexer (using only NAND gates), a four-bit parallel in Parallel-out shift register (utilizing D-flip-flops) and a 4-bit parallel-in serial-out shift register (also made using D-flip-flops). I aim to learn how to create and use multiplexers, understand how D-flipflops function and how to use them to create shift registers.







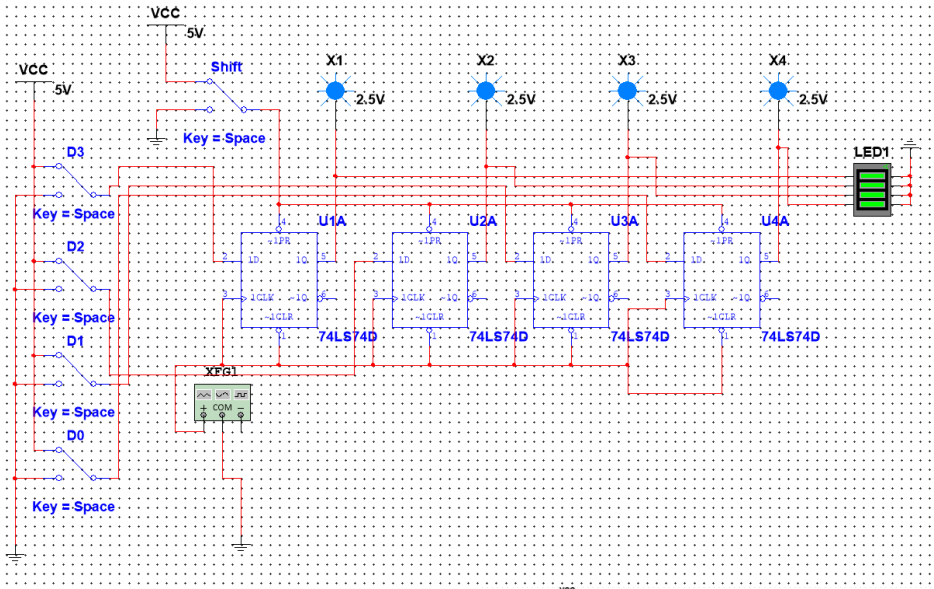
S0	S1	Input
0	0	D0
0	1	D1
1	0	D2
1	1	D3

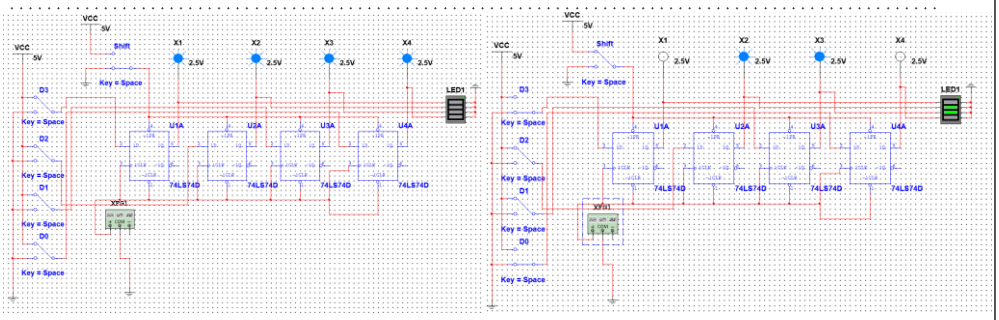
Task 1 – Four Input Multiplexer

The switches go into NAND gates which detect the input and (depending on the order of the key switches) output a result at the indicator. The NAND gates are connected so that the output depends on the combination of the bottom 2 switch inputs

I tested the outputs matched the truth table, which you can see from screenshots 2 and 3. I did this by setting all switches off, and then turning on the switch that is supposed to trigger an output. I also turned on all switches **except** the intended one to confirm that no other switches led to an output

s388590 Isaac Jessen





Task 2- Parallel-in Parallel-out shift registers

The Shift switch lets you load parallel data or control whether the register shifts

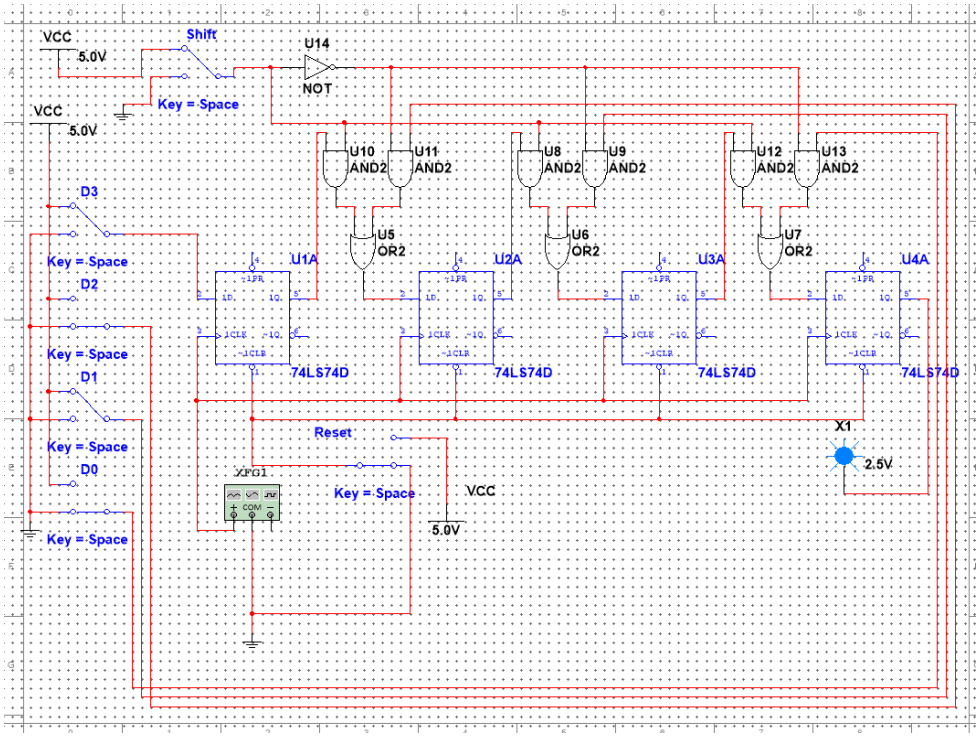
The flip-flops are all connected to a clock, so the voltage moves across all of them at once.

The circuit works in this order:

- set the parallel inputs
- use the Shift control to choose between loading or shifting
- Pulse the clock to watch the bits move through the flip-flops until they all appear at the LED bar

I spent a long time debugging the circuit before I realized that the LED bar does not accept output and ground wires both ways (I had attached the output wires to the wrong side of the LED bar).

s388590 Isaac Jessen



Task 3 – Parallel-in Serial-out shift register

For this task I copied task 2's circuit and added multiplexers between the flip-flops.

The reset switch resets the information inside the D-flip-flops.

When the Shift switch is set to low, on the next rising edge of the clock signal, the flip-flops store the inputted bits.

When this Shift switch is set to high, the parallel inputs are disabled.

Conclusion:

how multiplexers can route one of several input sources to a single output based on binary SELECT inputs

Through the three tasks I learned how NAND gates can be placed to route one of multiple inputs to a single output depending on a combination of switches. From the parallel-in parallel-out register I learned how D-flip-flops can store and present data simultaneously. Finally, the parallel-in serial-out register showed how control logic (using gates and a Shift signal) can be used to let data be loaded in parallel and then outputted bit by bit. I learned about how to create multiplexers, how they function and also how they can practically be used in parallel shift registers. I also learned the difference between parallel-out and serial-out shift registers, what they do and how they can be made.